

테이블 명세서 / ERD — 전문가 쿠폰 시스템 (M1)

항목	내용
문서 버전	v1.1 (PRD v1.2-M1 대응)
작성일	2026-09-12
DB	PostgreSQL
공통 규약	시각: UTC TIMESTAMP (초 단위). 금액: BIGINT (KRW 정수). 비율: NUMERIC(5,2). enum: VARCHAR + CHECK. ID 채번·FK 선언 여부·문자셋은 팀 내규
v1.0 → v1.1 변경	coupon_policies.end_reason 삭제(자동 전이 폐지) · coupon_policy_counters에 budget_exhausted_at/budget_notified_at 추가 · coupon_policy_user_counters 신설 · coupons.replaces_coupon_id → issue_group_id+issue_seq · coupon_usages.status에 RELEASED 추가, released_at 추가 · coupon_policy_events 자동 사건 타입 삭제 · coupon_usages (policy_id,user_id,status) 인덱스 삭제

1. ERD

ERD 다이어그램은 이 문서 마지막 페이지(부록)에 첨부되어 있습니다. 원본 mermaid 소스는 쿠폰_ERD.mermaid 참조.

관계 범례: 실선 = FK(선언 또는 논리), 점선 = 배치·검증·그룹 관계(FK 없음). 테이블 라벨은 한글명 (영문 테이블명), 컬럼 뒤 따옴표는 한글 설명이며 참조 대상 테이블·컬럼을 함께 적었다. UK 표기는 DB UNIQUE 제약이 있는 컬럼에만 붙였다. users · orders · order_items · refunds는 외부 도메인이며 참조 방향 표시용으로만 포함했다. 별도 파일 쿠폰_ERD.mermaid와 동일 내용.

1.1 키 요약 (PK / FK / 유일성)

FK 열의 선언은 DDL에 REFERENCES를 두는 것, 논리는 다형 참조·아카이브 참조·외부 도메인 등의 이유로 DDL에 두지 않고 애플리케이션이 보장하는 것이다. 외부 도메인(users, orders 등) FK의 선언 여부는 팀 내규를 따른다.

[DBA] FK 컬럼이 포함된 복합 유일성·부분 유일성은 DB UNIQUE 제약을 두지 않고 애플리케이션이 보장한다 (카운터/쿠폰 행 락 안에서 존재 검사 후 INSERT). DB UNIQUE는 FK가 아닌 단일 컬럼(issue_request_id)에만 둔다.

테이블	PK	FK (선언)	FK (논리)	DB UNIQUE	앱 보장 유일성
coupon_policies	id	—	issuer_id → users.id (EXPERT), created_by → users.id	—	—
coupon_policy_counters	policy_id	policy_id → coupon_policies.id	—	—	—
coupon_policy_user_counters	(policy_id, user_id)	policy_id → coupon_policies.id	user_id → users.id	—	—
coupon_policy_scopes	id	policy_id → coupon_policies.id	target_id → users/categories/products (다형)	—	(policy_id, target_type, target_id, mode)
coupon_policy_conditions	id	policy_id → coupon_policies.id	—	—	(policy_id, condition_type)
coupon_policy_histories	id	policy_id → coupon_policies.id	changed_by → users.id	—	(policy_id, version)
coupon_policy_events	id	policy_id → coupon_policies.id	actor_id → users.id	—	—
coupons	id	policy_id → coupon_policies.id	policy_version → histories(policy_id, version), user_id → users.id, issue_group_id → coupons/archives.id (꺾)	issue_request_id	(issue_group_id, issue_seq)
coupon_archives	id	policy_id → coupon_policies.id	policy_version → histories, user_id → users.id, issue_group_id (꺾)	issue_request_id	(issue_group_id, issue_seq)
coupon_usages	id	stack_slot → coupon_stacking_rules.stack_slot, policy_id → coupon_policies.id	coupon_id → coupons/archives, order_id → orders.id, user_id → users.id, policy_version → histories, issuer_id → users.id (EXPERT)	—	(coupon_id) WHERE APPLIED, (order_id, stack_slot) WHERE APPLIED
coupon_usage_items	id	usage_id → coupon_usages.id	order_item_id → order_items.id	—	(usage_id, order_item_id)
coupon_usage_refunds	id	usage_id → coupon_usages.id	refund_id → refunds.id	—	(usage_id, refund_id)
coupon_issuer_limits	issuer_type	—	updated_by → users.id	—	—

coupon_stacking_rules	stack_slot	issuer_type → coupon_issuer_limits.issuer_type	updated_by → users.id	—	(issuer_type)
coupon_setting_histories	id	—	changed_by → users.id	—	—
coupon_expert_user_usage_stats	(issuer_id, user_id)	—	issuer_id → users.id, user_id → users.id	—	—

2. 테이블 목록

#	테이블	그룹	갱신 빈도	보존	비고
1	coupon_policies	정책	낮음	영구	물리 삭제 금지. status는 수동 결정만
2	coupon_policy_counters	정책	높음	영구	발급·결제·취소·증액 시 행 락 대상. 예산 소진 시각 보유
3	coupon_policy_user_counters	정책	높음	영구	1인 발급/사용 한도. 정책 카운터 다음에 락
4	coupon_policy_scopes	정책	낮음	영구	
5	coupon_policy_conditions	정책	낮음	영구	
6	coupon_policy_histories	정책	낮음	5년+	version별 JSONB
7	coupon_policy_events	정책	낮음	5년+	수동 사건 감사 + 재발급/알림 큐
8	coupons	쿠폰	높음	아카이빙	종료 후 N일 뒤 이관
9	coupon_archives	쿠폰	배치	5년	coupons와 동일 컬럼 + archived_at
10	coupon_usages	사용	높음	5년	정산 소스 (RELEASED 제외)
11	coupon_usage_items	사용	높음	5년	스태킹 배분 근거
12	coupon_usage_refunds	사용	중간	5년	정산 조정 소스
13	coupon_issuer_limits	설정	매우 낮음	영구	행 수 = 발급 주체 수
14	coupon_stacking_rules	설정	매우 낮음	영구	행 수 = 슬롯 수
15	coupon_setting_histories	설정	매우 낮음	영구	
16	coupon_expert_user_usage_stats	집계	배치	영구	재생성 가능

3. 테이블 상세

3.1 coupon_policies — 쿠폰 정책 본문

컬럼	타입	NULL	기본값	설명
id	BIGINT	N		PK
issuer_type	VARCHAR(20)	N		PLATFORM / EXPERT
issuer_id	BIGINT	Y		issuer_type 별 다형 참조 (EXPERT → users.id). FK 미선언. PLATFORM이면 NULL
name	VARCHAR(100)	N		쿠폰명 (노출용)
description	VARCHAR(500)	Y		
discount_amount	BIGINT	Y		정액 할인(원). discount_rate와 정확히 하나만 NOT NULL
discount_rate	NUMERIC(5,2)	Y		비율 할인(%), 0.01~100.00
max_discount_amount	BIGINT	Y		최대 할인액
max_discount_ratio	NUMERIC(5,2)	Y		할인 전 총 주문 금액 대비 최대 할인 %
scope_type	VARCHAR(10)	N		ALL / TARGETED. TARGETED면 coupon_policy_scopes 행 1개 이상
validity_type	VARCHAR(10)	N		ABSOLUTE / RELATIVE
valid_from	TIMESTAMP	Y		ABSOLUTE 시작 (UTC)
valid_to	TIMESTAMP	Y		ABSOLUTE 종료 (UTC)
valid_days	INT	Y		RELATIVE: 발급일로부터 N일
issue_start_at	TIMESTAMP	N		발급 가능 시작 (UTC)
issue_end_at	TIMESTAMP	N		발급 가능 종료 (UTC). 경과 시 계산식으로 발급 불가. 연장은 UPDATE (상태 전이 없음)
issue_limit_total	INT	Y		총 발급 한도. NULL = 무제한
issue_limit_per_user	INT	Y		1인 발급 한도
use_limit_per_user	INT	Y		1인 사용 한도

stackable	BOOLEAN	N		중복 적용 허용
budget_amount	BIGINT	Y		예산. NULL = 무제한
status	VARCHAR(20)	N	'DRAFT'	수동 결정만: DRAFT / ACTIVE / PAUSED / ENDED / REVOKED
version	INT	N	1	정책 내용 변경 시에만 +1. 카운터·상태 전이는 증가시키지 않음
created_by	BIGINT	N		
created_at	TIMESTAMP	N	now()	
updated_at	TIMESTAMP	N	now()	

판정식 (저장하지 않음)

```
issuable = status = 'ACTIVE'
          AND issue_start_at <= now() AND now() < issue_end_at
          AND counters.budget_exhausted_at IS NULL
          AND (issue_limit_total IS NULL OR counters.issued_count < issue_limit_total)
redeemable = status <> 'REVOKED'
active_for_limit = status IN ('ACTIVE','PAUSED')          -- 동시 활성 정책 수 집계
```

목록 탭 (SQL CASE)

탭	조건
예정	status='ACTIVE' AND now() < issue_start_at
진행 중	issuable 또는 status='PAUSED' AND now() < issue_end_at (일시 중지 뱃지)
종료	그 외 (ENDED, REVOKED, 기간 경과, 예산 소진, 총 한도 소진) — 뱃지로 원인 표시

제약

- CHECK ((discount_amount IS NOT NULL) <> (discount_rate IS NOT NULL))
- CHECK (discount_amount IS NULL OR discount_amount > 0)
- CHECK (discount_rate IS NULL OR (discount_rate >= 0.01 AND discount_rate <= 100))
- CHECK (max_discount_ratio IS NULL OR (max_discount_ratio > 0 AND max_discount_ratio <= 100))
- CHECK (validity_type = 'ABSOLUTE' AND valid_from IS NOT NULL AND valid_to IS NOT NULL AND valid_days IS NULL OR validity_type = 'RELATIVE' AND valid_days IS NOT NULL AND valid_from IS NULL AND valid_to IS NULL)
- CHECK (issuer_type <> 'EXPERT' OR (issuer_id IS NOT NULL AND scope_type = 'TARGETED'))
- CHECK (issue_start_at < issue_end_at)

인덱스

이름	컬럼	용도
ix_cp_issuer_status	(issuer_type, issuer_id, status)	전문가 목록, 동시 활성 수 검사
ix_cp_status_issue_end	(status, issue_end_at)	목록 탭 계산·기간 경과 뱃지

3.2 coupon_policy_counters — 정책 카운터 (1:1)

컬럼	타입	NULL	기본값	설명
policy_id	BIGINT	N		PK, FK → coupon_policies.id
issued_count	INT	N	0	발급 수 (재발급분 제외)
used_budget_amount	BIGINT	N	0	누적 사용 예산 (APPLIED 합)
budget_exhausted_at	TIMESTAMP	Y		예산 소진 시각. 차감 UPDATE에서 used + Δ >= budget 이 되는 순간 기록. 복원·증액으로 used < budget 이 되면 NULL
budget_notified_at	TIMESTAMP	Y		소진 알림 발송 시각. 커밋 후 혹 또는 보완 배치가 기록. 복원 시 NULL
updated_at	TIMESTAMP	N	now()	

갱신 규칙 (모두 이 행 락 안에서 한 문장)

```

-- 발급 (총 한도)
UPDATE coupon_policy_counters c SET issued_count = issued_count + 1, updated_at = now()
FROM coupon_policies p WHERE c.policy_id = p.id AND c.policy_id = :pid
AND (p.issue_limit_total IS NULL OR c.issued_count < p.issue_limit_total);

-- 결제 확정 (예산 차감 + 소진 시각)
UPDATE coupon_policy_counters c
SET used_budget_amount = c.used_budget_amount + :d,
    budget_exhausted_at = CASE WHEN p.budget_amount IS NOT NULL AND c.used_budget_amount + :d >= p.budget_amount
                              THEN now() ELSE c.budget_exhausted_at END,
    updated_at = now()
FROM coupon_policies p WHERE c.policy_id = p.id AND c.policy_id = :pid
AND (p.budget_amount IS NULL OR c.used_budget_amount + :d <= p.budget_amount);
-- 갱신 0행이면 잔여 예산으로 재계산 1회 재시도, 그래도 0행이면 결제 거부

-- 취소 / 결제 실패 (복원) 및 예산 증액
UPDATE coupon_policy_counters c
SET used_budget_amount = GREATEST(c.used_budget_amount - :d, 0),
    budget_exhausted_at = CASE WHEN p.budget_amount IS NULL OR c.used_budget_amount - :d < p.budget_amount
                              THEN NULL ELSE c.budget_exhausted_at END,
    budget_notified_at = CASE WHEN p.budget_amount IS NULL OR c.used_budget_amount - :d < p.budget_amount
                              THEN NULL ELSE c.budget_notified_at END,
    updated_at = now()
FROM coupon_policies p WHERE c.policy_id = p.id AND c.policy_id = :pid;
-- 예산 증액은 :d = 0 으로 같은 문장 실행

```

3.3 coupon_policy_user_counters — 정책·회원 카운터

컬럼	타입	NULL	기본값	설명
policy_id	BIGINT	N		PK(복합), FK → coupon_policies.id
user_id	BIGINT	N		PK(복합), users.id (논리 FK)
issued_count	INT	N	0	1인 발급 수 (재발급분 제외)
used_count	INT	N	0	1인 사용 수. reserve +1, cancel/release -1
updated_at	TIMESTAMP	N	now()	

규칙

- 첫 발급 시 INSERT ... ON CONFLICT (policy_id, user_id) DO UPDATE SET issued_count = issued_count + 1 WHERE issued_count < :limit (한도 NULL 이면 조건 생략).
- 사용 한도: reserve에서 UPDATE ... SET used_count = used_count + 1 WHERE used_count < :limit.
- 락 순서: coupon_policy_counters → coupon_policy_user_counters (발급·reserve 모두). 역순 금지.

3.4 coupon_policy_scopes — 적용 범위

컬럼	타입	NULL	기본값	설명
id	BIGINT	N		PK
policy_id	BIGINT	N		FK → coupon_policies.id
target_type	VARCHAR(20)	N		EXPERT / CATEGORY / PRODUCT
target_id	BIGINT	N		대상 ID (EXPERT: users.id, CATEGORY: categories.id, PRODUCT: products.id)
mode	VARCHAR(10)	N		INCLUDE / EXCLUDE

- 유일성 (policy_id, target_type, target_id, mode): 앱 보장
- ix_cps_policy (policy_id)
- ix_cps_target (target_type, target_id) — 상품 상세 역조회 (상품 ID, 카테고리 조상 목록(categories.path 전제), 전문가 ID 세 종류 IN)

3.5 coupon_policy_conditions — 사용 조건

컬럼	타입	NULL	기본값	설명
id	BIGINT	N		PK
policy_id	BIGINT	N		FK → coupon_policies.id
condition_type	VARCHAR(50)	N		레지스트리 키. M1: MIN_ORDER_AMOUNT
params	JSONB	N		타입별 파라미터. 저장 시 JSON Schema 검증
sort_order	INT	N	0	평가 순서

- 유일성 (policy_id, condition_type): 앱 보장

- ix_cpc_policy (policy_id)

3.6 coupon_policy_histories — 정책 버전 이력

컬럼	타입	NULL	기본값	설명
id	BIGINT	N		PK
policy_id	BIGINT	N		FK → coupon_policies.id
version	INT	N		coupon_policies.version과 동일 값
content	JSONB	N		{ "policy": {...}, "scopes": [...], "conditions": [...] } (PRD § 7.4)
changed_by	BIGINT	N		
created_at	TIMESTAMP	N	now()	

- 유일성 (policy_id, version): 앱 보장
- ix_cph_policy_version (policy_id, version)
- 생성 시 version 1 기록, 내용 변경마다 새 행. 삭제 금지.

3.7 coupon_policy_events — 수동 사건 감사 로그 / 배치 큐

컬럼	타입	NULL	기본값	설명
id	BIGINT	N		PK
policy_id	BIGINT	N		FK → coupon_policies.id
event_type	VARCHAR(30)	N		아래 목록 (수동 사건만)
actor_type	VARCHAR(10)	N		EXPERT / ADMIN
actor_id	BIGINT	N		users.id
reason	VARCHAR(500)	Y		강제 중지 사유 등
detail	JSONB	Y		변경 전후 값
processed_at	TIMESTAMP	Y		후속 배치 처리 완료 시각. 처리 불필요 이벤트는 생성 시 채움
notified_at	TIMESTAMP	Y		알림 발송 완료 시각. 알림 불필요 이벤트는 생성 시 채움
created_at	TIMESTAMP	N	now()	

event_type

값	발생	후속 배치	알림
CREATED / UPDATED	생성 / 내용 변경 (version +1)	-	-
ACTIVATED / PAUSED / RESUMED / ENDED	수동 상태 전이	-	-
REVOKED	운영자 회수	-	전문가
VALIDITY_EXTENDED	유효기간 연장	재발급 배치	-
ISSUE_PERIOD_EXTENDED	발급 기간 연장 (issue_end_at)	-	-
LIMIT_INCREASED / BUDGET_INCREASED	한도·예산 증액	-	-

예산 소진·복원, 발급 기간 경과는 이벤트 행을 만들지 않는다 (카운터 시각·계산식으로 표현).

인덱스

- ix_cpe_policy (policy_id, created_at)
- ix_cpe_pending_process (event_type) WHERE processed_at IS NULL (partial)
- ix_cpe_pending_notify (created_at) WHERE notified_at IS NULL (partial)

3.8 coupons — 발급 쿠폰

컬럼	타입	NULL	기본값	설명
id	BIGINT	N		PK
policy_id	BIGINT	N		FK → coupon_policies.id
policy_version	INT	N		발급 당시 version (histories 논리 FK)
user_id	BIGINT	N		소유 의뢰인 (users.id)

issue_group_id	BIGINT	N		발급 그룹 ID. 최초 발급 시 자기 id 값, 재발급 시 물려받음. FK 없음 (아카이브된 행을 가리켜도 무방)
issue_seq	SMALLINT	N	1	그룹 내 순번. 최초 1, 재발급마다 +1. 현재 유효본 = 그룹 내 max
status	VARCHAR(20)	N	'ISSUED'	ISSUED / USED / EXPIRED / REPLACED (REVOKED 없음 — 정책 상태로 lazy 판단)
issue_request_id	VARCHAR(64)	N		먹통키. 클라이언트 UUID / 재발급은 REISSUE:{issue_group_id}:{issue_seq}
issued_at	TIMESTAMP	N	now()	
expires_at	TIMESTAMP	N		ABSOLUTE: valid_to / RELATIVE: issued_at + valid_days. 갱신하지 않음 (연장은 재발급)
used_at	TIMESTAMP	Y		
created_at	TIMESTAMP	N	now()	
updated_at	TIMESTAMP	N	now()	

제약·인덱스

- UNIQUE (issue_request_id) — 먹통
- 유일성 (issue_group_id, issue_seq): 앱 보장 (재발급 먹통키가 사실상 보장)
- ix_c_user_status (user_id, status, expires_at) — 내 쿠폰 목록
- ix_c_group (issue_group_id, issue_seq) — 그룹 조회, 최신본
- ix_c_status_expires (status, expires_at) — 만료 배치, 재발급 배치
- ix_c_status_updated (status, updated_at) — 아카이빙 배치
- ※ 1인 발급 한도는 coupon_policy_user_counters로 검사하므로 (policy_id, user_id) 인덱스 불필요

3.9 coupon_archives — 쿠폰 아카이브

coupons와 동일 컬럼 + archived_at TIMESTAMP NOT NULL. PK id (원본 ID 유지). 인덱스 (user_id, archived_at), (issue_group_id, issue_seq), (policy_id). 이관 대상: status IN ('USED', 'EXPIRED', 'REPLACED') AND updated_at < now() - N일 (N은 BE 회의에서 확정, 제안 90). 그룹 참조가 값이므로 이관 순서·FK 충돌 없음. coupon_usages.coupon_id는 두 테이블 중 하나를 가리키므로 FK 미선언.

3.10 coupon_usages — 사용 이력

컬럼	타입	NULL	기본값	설명
id	BIGINT	N		PK
coupon_id	BIGINT	N		논리 FK (아카이브 참조 가능). APPLIED 1건 유일성 앱 보장
order_id	BIGINT	N		orders.id
stack_slot	SMALLINT	N		FK → coupon_stacking_rules.stack_slot. (order_id, stack_slot) APPLIED 유일성 앱 보장
user_id	BIGINT	N		비정규화 (반복 사용 집계)
policy_id	BIGINT	N		비정규화. FK → coupon_policies.id
policy_version	INT	N		적용 시점 version (histories 논리 FK)
issuer_type	VARCHAR(20)	N		비정규화 (정산)
issuer_id	BIGINT	Y		비정규화 (정산)
order_total_amount	BIGINT	N		할인 전 총 주문 금액
eligible_amount	BIGINT	N		적용 대상 금액 (원금액)
base_amount	BIGINT	N		차감 기준 금액 (앞 슬롯 반영 후)
discount_amount	BIGINT	N		최종 할인액
status	VARCHAR(20)	N	'APPLIED'	APPLIED / RELEASED (결제 실패, 정산 제외) / CANCELLED (주문 취소, 정산 조정). 제자리 UPDATE
applied_at	TIMESTAMP	N	now()	정산 회차 기준
released_at	TIMESTAMP	Y		결제 실패 해제 시각
cancelled_at	TIMESTAMP	Y		주문 취소 시각 (정산 조정 회차 기준)

인덱스

- ix_cu_coupon (coupon_id, status) — 이중 사용 존재 검사
- ix_cu_order (order_id, stack_slot) — 슬롯 중복 존재 검사, reserve 먹통 조회
- ix_cu_settlement (issuer_type, issuer_id, applied_at) WHERE status = 'APPLIED' — 정산 pull
- ix_cu_cancelled (cancelled_at) WHERE cancelled_at IS NOT NULL — 정산 조정
- ix_cu_issuer_user (issuer_id, user_id) — 반복 사용 집계
- ix_cu_policy_status (policy_id, status) — 카운터 검증·재계산

3.11 coupon_usage_items — 항목별 할인 배분 (스태킹 근거)

컬럼	타입	NULL	기본값	설명
id	BIGINT	N		PK
usage_id	BIGINT	N		FK → coupon_usages.id
order_item_id	BIGINT	N		주문 항목 ID
item_amount	BIGINT	N		항목 원금액 (기본가 + 옵션가)
allocated_discount_amount	BIGINT	N		이 쿠폰이 이 항목에 배분한 할인액. 잔액은 금액이 큰 항목에 귀속

- 유일성 (usage_id, order_item_id): 앱 보장
- ix_cui_usage (usage_id)
- 용도: 뒤 슬롯 base_amount 산출 근거. **환불 배분에는 쓰지 않는다** (환불은 금액 단위).

3.12 coupon_usage_refunds — 부분 환불 배분 이력 (금액 단위)

컬럼	타입	NULL	기본값	설명
id	BIGINT	N		PK
usage_id	BIGINT	N		FK → coupon_usages.id
refund_id	BIGINT	N		주문 도메인 환불 ID
refunded_discount_amount	BIGINT	N		floor(discount_amount × R / 실결제 금액) (원 단위 버림)
refunded_at	TIMESTAMP	N		정산 조정 회차 기준
created_at	TIMESTAMP	N	now()	

- 유일성 (usage_id, refund_id): 앱 보장
- ix_cur_usage (usage_id), ix_cur_refunded_at (refunded_at)
- 환불 사건 1건 = 쿠폰당 1행. 누적 컬럼을 두지 않는 이유: 정산이 refunded_at ∈ 기간으로 조정 회차를 잡아야 함.

3.13 coupon_issuer_limits — 발급 주체별 제한

컬럼	타입	NULL	기본값	설명
issuer_type	VARCHAR(20)	N		PK. PLATFORM / EXPERT
max_discount_rate	NUMERIC(5,2)	Y		최대 비율 할인 %. NULL = 제한 없음
max_active_policies	INT	Y		동시 활성(ACTIVE + PAUSED) 정책 수
min_valid_days	INT	Y		
max_valid_days	INT	Y		
max_issue_period_days	INT	Y		발급 기간 상한
default_stackable	BOOLEAN	N		신규 정책 기본값
updated_by	BIGINT	N		
updated_at	TIMESTAMP	N	now()	

초기 데이터 [결정값]: EXPERT (90.00, 20, 1, 180, 365, true) / PLATFORM (NULL, NULL, NULL, NULL, NULL, false)

3.14 coupon_stacking_rules — 스태킹 슬롯

컬럼	타입	NULL	기본값	설명
stack_slot	SMALLINT	N		PK. 적용 순서
issuer_type	VARCHAR(20)	N		FK → coupon_issuer_limits.issuer_type (유일성 앱 보장)
enabled	BOOLEAN	N	true	false면 해당 주체 쿠폰은 단독 사용만 가능
updated_by	BIGINT	N		
updated_at	TIMESTAMP	N	now()	

초기 데이터: (1, 'PLATFORM', true), (2, 'EXPERT', true). 주문당 최대 쿠폰 수 = enabled 행 수.

3.15 coupon_setting_histories — 설정 변경 이력

컬럼	타입	NULL	기본값	설명
id	BIGINT	N		PK
setting_table	VARCHAR(50)	N		coupon_issuer_limits / coupon_stacking_rules
setting_key	VARCHAR(50)	N		행 식별자 (issuer_type 또는 stack_slot)
before_value	JSONB	Y		
after_value	JSONB	N		
changed_by	BIGINT	N		
changed_at	TIMESTAMP	N	now()	

- ix_csh_table (setting_table, changed_at)

3.16 coupon_expert_user_usage_stats — 반복 사용 집계

컬럼	타입	NULL	기본값	설명
issuer_id	BIGINT	N		PK (복합). 전문가
user_id	BIGINT	N		PK (복합). 의뢰인
used_count	INT	N		APPLIED 사용 횟수
last_used_at	TIMESTAMP	N		
updated_at	TIMESTAMP	N	now()	집계 배치 갱신 시각

- ix_ceus_count (used_count DESC)
- 배치: SELECT issuer_id, user_id, count(*), max(applied_at) FROM coupon_usages WHERE issuer_type='EXPERT' AND status='APPLIED' GROUP BY 1,2 → upsert. 전량 재생성 가능.

4. DDL (PostgreSQL)

```
-- 3.1
CREATE TABLE coupon_policies (
    id                BIGINT PRIMARY KEY,
    issuer_type       VARCHAR(20) NOT NULL,
    issuer_id         BIGINT,
    name              VARCHAR(100) NOT NULL,
    description        VARCHAR(500),
    discount_amount    BIGINT,
    discount_rate      NUMERIC(5,2),
    max_discount_amount BIGINT,
    max_discount_ratio NUMERIC(5,2),
    scope_type         VARCHAR(10) NOT NULL,
    validity_type      VARCHAR(10) NOT NULL,
    valid_from         TIMESTAMP,
    valid_to           TIMESTAMP,
    valid_days         INT,
    issue_start_at     TIMESTAMP NOT NULL,
    issue_end_at       TIMESTAMP NOT NULL,
    issue_limit_total  INT,
    issue_limit_per_user INT,
    use_limit_per_user INT,
    stackable          BOOLEAN NOT NULL,
    budget_amount      BIGINT,
    status             VARCHAR(20) NOT NULL DEFAULT 'DRAFT',
    version            INT NOT NULL DEFAULT 1,
    created_by         BIGINT NOT NULL,
    created_at         TIMESTAMP NOT NULL DEFAULT now(),
    updated_at         TIMESTAMP NOT NULL DEFAULT now(),
    CONSTRAINT ck_cp_issuer_type CHECK (issuer_type IN ('PLATFORM', 'EXPERT')),
    CONSTRAINT ck_cp_discount_xor CHECK ((discount_amount IS NOT NULL) <> (discount_rate IS NOT NULL)),
    CONSTRAINT ck_cp_discount_amt CHECK (discount_amount IS NULL OR discount_amount > 0),
    CONSTRAINT ck_cp_discount_rate CHECK (discount_rate IS NULL OR (discount_rate >= 0.01 AND discount_rate <= 100)),
    CONSTRAINT ck_cp_max_ratio CHECK (max_discount_ratio IS NULL OR (max_discount_ratio > 0 AND max_discount_ratio <= 100)),
    CONSTRAINT ck_cp_scope_type CHECK (scope_type IN ('ALL', 'TARGETED')),
    CONSTRAINT ck_cp_validity CHECK (
        (validity_type = 'ABSOLUTE' AND valid_from IS NOT NULL AND valid_to IS NOT NULL AND valid_days IS NULL)
        OR (validity_type = 'RELATIVE' AND valid_days IS NOT NULL AND valid_from IS NULL AND valid_to IS NULL)),
    CONSTRAINT ck_cp_expert_scope CHECK (issuer_type <> 'EXPERT' OR (issuer_id IS NOT NULL AND scope_type = 'TARGETED')),
    CONSTRAINT ck_cp_status CHECK (status IN ('DRAFT', 'ACTIVE', 'PAUSED', 'ENDED', 'REVOKED')),
    CONSTRAINT ck_cp_issue_period CHECK (issue_start_at < issue_end_at)
);
CREATE INDEX ix_cp_issuer_status ON coupon_policies (issuer_type, issuer_id, status);
CREATE INDEX ix_cp_status_issue_end ON coupon_policies (status, issue_end_at);

-- 3.2
CREATE TABLE coupon_policy_counters (
    policy_id        BIGINT PRIMARY KEY REFERENCES coupon_policies(id),
    issued_count     INT NOT NULL DEFAULT 0,
```

```

        used_budget_amount BIGINT NOT NULL DEFAULT 0,
        budget_exhausted_at TIMESTAMP,
        budget_notified_at TIMESTAMP,
        updated_at TIMESTAMP NOT NULL DEFAULT now(),
        CONSTRAINT ck_cpc_nonneg CHECK (issued_count >= 0 AND used_budget_amount >= 0)
    );
    CREATE INDEX ix_cpc_budget_notify ON coupon_policy_counters (budget_exhausted_at)
        WHERE budget_exhausted_at IS NOT NULL AND budget_notified_at IS NULL;

-- 3.3
CREATE TABLE coupon_policy_user_counters (
    policy_id BIGINT NOT NULL REFERENCES coupon_policies(id),
    user_id BIGINT NOT NULL,
    issued_count INT NOT NULL DEFAULT 0,
    used_count INT NOT NULL DEFAULT 0,
    updated_at TIMESTAMP NOT NULL DEFAULT now(),
    PRIMARY KEY (policy_id, user_id),
    CONSTRAINT ck_cpuc_nonneg CHECK (issued_count >= 0 AND used_count >= 0)
);

-- 3.4
CREATE TABLE coupon_policy_scopes (
    id BIGINT PRIMARY KEY,
    policy_id BIGINT NOT NULL REFERENCES coupon_policies(id),
    target_type VARCHAR(20) NOT NULL,
    target_id BIGINT NOT NULL,
    mode VARCHAR(10) NOT NULL,
    CONSTRAINT ck_cps_target_type CHECK (target_type IN ('EXPERT', 'CATEGORY', 'PRODUCT')),
    CONSTRAINT ck_cps_mode CHECK (mode IN ('INCLUDE', 'EXCLUDE'))
);
CREATE INDEX ix_cps_policy ON coupon_policy_scopes (policy_id);
CREATE INDEX ix_cps_target ON coupon_policy_scopes (target_type, target_id);

-- 3.5
CREATE TABLE coupon_policy_conditions (
    id BIGINT PRIMARY KEY,
    policy_id BIGINT NOT NULL REFERENCES coupon_policies(id),
    condition_type VARCHAR(50) NOT NULL,
    params JSONB NOT NULL,
    sort_order INT NOT NULL DEFAULT 0
);
CREATE INDEX ix_cpc_policy ON coupon_policy_conditions (policy_id);

-- 3.6
CREATE TABLE coupon_policy_histories (
    id BIGINT PRIMARY KEY,
    policy_id BIGINT NOT NULL REFERENCES coupon_policies(id),
    version INT NOT NULL,
    content JSONB NOT NULL,
    changed_by BIGINT NOT NULL,
    created_at TIMESTAMP NOT NULL DEFAULT now()
);
CREATE INDEX ix_cph_policy_version ON coupon_policy_histories (policy_id, version);

-- 3.7
CREATE TABLE coupon_policy_events (
    id BIGINT PRIMARY KEY,
    policy_id BIGINT NOT NULL REFERENCES coupon_policies(id),
    event_type VARCHAR(30) NOT NULL,
    actor_type VARCHAR(10) NOT NULL,
    actor_id BIGINT NOT NULL,
    reason VARCHAR(500),
    detail JSONB,
    processed_at TIMESTAMP,
    notified_at TIMESTAMP,
    created_at TIMESTAMP NOT NULL DEFAULT now(),
    CONSTRAINT ck_cpe_event_type CHECK (event_type IN (
        'CREATED', 'UPDATED', 'ACTIVATED', 'PAUSED', 'RESUMED', 'ENDED', 'REVOKED',
        'VALIDITY_EXTENDED', 'ISSUE_PERIOD_EXTENDED', 'LIMIT_INCREASED', 'BUDGET_INCREASED')),
    CONSTRAINT ck_cpe_actor_type CHECK (actor_type IN ('EXPERT', 'ADMIN'))
);
CREATE INDEX ix_cpe_policy ON coupon_policy_events (policy_id, created_at);
CREATE INDEX ix_cpe_pending_process ON coupon_policy_events (event_type) WHERE processed_at IS NULL;
CREATE INDEX ix_cpe_pending_notify ON coupon_policy_events (created_at) WHERE notified_at IS NULL;

-- 3.8
CREATE TABLE coupons (
    id BIGINT PRIMARY KEY,
    policy_id BIGINT NOT NULL REFERENCES coupon_policies(id),
    policy_version INT NOT NULL,
    user_id BIGINT NOT NULL,
    issue_group_id BIGINT NOT NULL, -- 값 참조, FK 없음
    issue_seq SMALLINT NOT NULL DEFAULT 1,
    status VARCHAR(20) NOT NULL DEFAULT 'ISSUED',
    issue_request_id VARCHAR(64) NOT NULL,
    issued_at TIMESTAMP NOT NULL DEFAULT now(),
    expires_at TIMESTAMP NOT NULL,
    used_at TIMESTAMP,
    created_at TIMESTAMP NOT NULL DEFAULT now(),
    updated_at TIMESTAMP NOT NULL DEFAULT now(),
    CONSTRAINT ck_c_status CHECK (status IN ('ISSUED', 'USED', 'EXPIRED', 'REPLACED')),
    CONSTRAINT ck_c_issue_seq CHECK (issue_seq >= 1),
    CONSTRAINT uq_c_issue_request UNIQUE (issue_request_id)
);
CREATE INDEX ix_c_user_status ON coupons (user_id, status, expires_at);
CREATE INDEX ix_c_group ON coupons (issue_group_id, issue_seq);
CREATE INDEX ix_c_status_expires ON coupons (status, expires_at);

```

```

CREATE INDEX ix_c_status_updated ON coupons (status, updated_at);

-- 3.9
CREATE TABLE coupon_archives (LIKE coupons INCLUDING DEFAULTS);
ALTER TABLE coupon_archives ADD COLUMN archived_at TIMESTAMP NOT NULL;
ALTER TABLE coupon_archives ADD PRIMARY KEY (id);
ALTER TABLE coupon_archives ADD CONSTRAINT uq_ca_issue_request UNIQUE (issue_request_id);
CREATE INDEX ix_ca_user ON coupon_archives (user_id, archived_at);
CREATE INDEX ix_ca_group ON coupon_archives (issue_group_id, issue_seq);
CREATE INDEX ix_ca_policy ON coupon_archives (policy_id);

-- 3.13 (3.10 이 stack_slot FK 로 3.14 를 참조하므로 설정 테이블을 먼저 생성)
CREATE TABLE coupon_issuer_limits (
    issuer_type VARCHAR(20) PRIMARY KEY,
    max_discount_rate NUMERIC(5,2),
    max_active_policies INT,
    min_valid_days INT,
    max_valid_days INT,
    max_issue_period_days INT,
    default_stackable BOOLEAN NOT NULL,
    updated_by BIGINT NOT NULL,
    updated_at TIMESTAMP NOT NULL DEFAULT now()
);
INSERT INTO coupon_issuer_limits VALUES
    ('EXPERT', 90.00, 20, 1, 180, 365, true, 0, now()),
    ('PLATFORM', NULL, NULL, NULL, NULL, NULL, false, 0, now());

-- 3.14
CREATE TABLE coupon_stacking_rules (
    stack_slot SMALLINT PRIMARY KEY,
    issuer_type VARCHAR(20) NOT NULL REFERENCES coupon_issuer_limits(issuer_type), -- 유일성 앱 보장
    enabled BOOLEAN NOT NULL DEFAULT true,
    updated_by BIGINT NOT NULL,
    updated_at TIMESTAMP NOT NULL DEFAULT now()
);
INSERT INTO coupon_stacking_rules VALUES (1, 'PLATFORM', true, 0, now()), (2, 'EXPERT', true, 0, now());

-- 3.10
CREATE TABLE coupon_usages (
    id BIGINT PRIMARY KEY,
    coupon_id BIGINT NOT NULL, -- 논리 FK (아카이브 참조 가능)
    order_id BIGINT NOT NULL,
    stack_slot SMALLINT NOT NULL REFERENCES coupon_stacking_rules(stack_slot),
    user_id BIGINT NOT NULL,
    policy_id BIGINT NOT NULL REFERENCES coupon_policies(id),
    policy_version INT NOT NULL,
    issuer_type VARCHAR(20) NOT NULL,
    issuer_id BIGINT, -- 다형 참조, FK 미선언
    order_total_amount BIGINT NOT NULL,
    eligible_amount BIGINT NOT NULL,
    base_amount BIGINT NOT NULL,
    discount_amount BIGINT NOT NULL,
    status VARCHAR(20) NOT NULL DEFAULT 'APPLIED',
    applied_at TIMESTAMP NOT NULL DEFAULT now(),
    released_at TIMESTAMP,
    cancelled_at TIMESTAMP,
    CONSTRAINT ck_cu_status CHECK (status IN ('APPLIED', 'RELEASED', 'CANCELLED')),
    CONSTRAINT ck_cu_amounts CHECK (discount_amount >= 0 AND discount_amount <= base_amount AND base_amount <= eligible_amount)
);
-- 유일성 (coupon_id WHERE APPLIED), (order_id, stack_slot WHERE APPLIED) 는 애플리케이션 보장
CREATE INDEX ix_cu_coupon ON coupon_usages (coupon_id, status);
CREATE INDEX ix_cu_order ON coupon_usages (order_id, stack_slot);
CREATE INDEX ix_cu_settlement ON coupon_usages (issuer_type, issuer_id, applied_at) WHERE status = 'APPLIED';
CREATE INDEX ix_cu_cancelled ON coupon_usages (cancelled_at) WHERE cancelled_at IS NOT NULL;
CREATE INDEX ix_cu_issuer_user ON coupon_usages (issuer_id, user_id);
CREATE INDEX ix_cu_policy_status ON coupon_usages (policy_id, status);

-- 3.11
CREATE TABLE coupon_usage_items (
    id BIGINT PRIMARY KEY,
    usage_id BIGINT NOT NULL REFERENCES coupon_usages(id),
    order_item_id BIGINT NOT NULL,
    item_amount BIGINT NOT NULL,
    allocated_discount_amount BIGINT NOT NULL,
    CONSTRAINT ck_cui_amount CHECK (allocated_discount_amount >= 0 AND allocated_discount_amount <= item_amount)
);
CREATE INDEX ix_cui_usage ON coupon_usage_items (usage_id);

-- 3.12
CREATE TABLE coupon_usage_refunds (
    id BIGINT PRIMARY KEY,
    usage_id BIGINT NOT NULL REFERENCES coupon_usages(id),
    refund_id BIGINT NOT NULL,
    refunded_discount_amount BIGINT NOT NULL,
    refunded_at TIMESTAMP NOT NULL,
    created_at TIMESTAMP NOT NULL DEFAULT now(),
    CONSTRAINT ck_cur_amount CHECK (refunded_discount_amount >= 0)
);
CREATE INDEX ix_cur_usage ON coupon_usage_refunds (usage_id);
CREATE INDEX ix_cur_refunded_at ON coupon_usage_refunds (refunded_at);

-- 3.15
CREATE TABLE coupon_setting_histories (
    id BIGINT PRIMARY KEY,
    setting_table VARCHAR(50) NOT NULL,
    setting_key VARCHAR(50) NOT NULL,
    before_value JSONB,

```

```

        after_value    JSONB      NOT NULL,
        changed_by    BIGINT     NOT NULL,
        changed_at     TIMESTAMP NOT NULL DEFAULT now()
    );
    CREATE INDEX ix_csh_table ON coupon_setting_histories (setting_table, changed_at);

-- 3.16
CREATE TABLE coupon_expert_user_usage_stats (
    issuer_id    BIGINT     NOT NULL,
    user_id      BIGINT     NOT NULL,
    used_count    INT       NOT NULL,
    last_used_at  TIMESTAMP NOT NULL,
    updated_at    TIMESTAMP NOT NULL DEFAULT now(),
    PRIMARY KEY (issuer_id, user_id)
);
CREATE INDEX ix_ceus_count ON coupon_expert_user_usage_stats (used_count DESC);

```

5. 주요 접근 패턴과 인덱스 대응

패턴	쿼리 형태	인덱스
상품 상세 받을 수 있는 쿠폰	scopes WHERE (target_type,target_id) IN (('PRODUCT',p),('CATEGORY',c1.cn),('EXPERT',e)) → 정책 issuable 계산 → 회원 카운터로 이미 받았는지	ix_cps_target, ix_cp_issuer_status, PK counters/user_counters
발급 (역등 + 한도)	coupons WHERE issue_request_id=? → UPDATE policy_counters ... (총 한도) → INSERT ... ON CONFLICT user_counters (1인 한도) → INSERT coupons	uq_c_issue_request, PK counters, PK user_counters
내 쿠폰 목록	coupons WHERE user_id=? AND status='ISSUED' ORDER BY expires_at + policies·counters 조인(REVOKED, 예산 소진)	ix_c_user_status
결제 evaluate (primary)	위 목록 + counters + scopes + conditions + stacking_rules	PK 접근
결제 reserve	usages WHERE order_id=? (역등) → coupons FOR UPDATE → APPLIED 존재 검사 → UPDATE policy_counters (예산+소진 시각) → UPDATE user_counters (사용 한도) → INSERT usages/items	ix_cu_order, PK, ix_cu_coupon
취소 / 결제 실패	UPDATE usages SET status, cancelled_at/released_at → 쿠폰 복구 → UPDATE policy_counters (복원+시각 NULL) → UPDATE user_counters used_count-1	ix_cu_order, PK
정산 pull	usages WHERE issuer_type=? AND applied_at ∈ [a,b) AND status='APPLIED' + refunds WHERE refunded_at ∈ [a,b) + usages WHERE cancelled_at ∈ [a,b)	ix_cu_settlement, ix_cur_refunded_at, ix_cu_cancelled
목록 탭	policies JOIN counters + CASE(status, issue_start/end, budget_exhausted_at, issued_count)	ix_cp_issuer_status
만료 배치	UPDATE coupons SET status='EXPIRED' WHERE status='ISSUED' AND expires_at < now() (체크)	ix_c_status_expires
재발급 배치	events WHERE event_type='VALIDITY_EXTENDED' AND processed_at IS NULL → coupons WHERE policy_id=? AND status='ISSUED' AND expires_at > now() → INSERT (group, seq+1)	ix_cpe_pending_process, ix_c_status_expires, ix_c_group
알림 보완 배치	counters WHERE budget_exhausted_at IS NOT NULL AND budget_notified_at IS NULL + events WHERE notified_at IS NULL	ix_cpc_budget_notify, ix_cpe_pending_notify
반복 사용 집계	usages WHERE issuer_type='EXPERT' AND status='APPLIED' GROUP BY issuer_id,user_id	ix_cu_issuer_user
아카이빙	coupons WHERE status IN (...) AND updated_at < now()-N → INSERT archives, DELETE	ix_c_status_updated

6. 후속 마일스톤 확장 지점 (참고)

기능	추가 테이블	기존 테이블 변경
CODE 발급	coupon_codes(policy_id, code, code_type, max_redemptions, redeemed_count, status)	coupons.code_id nullable 컬럼 추가
AUTO 발급	coupon_policy_triggers(policy_id, event_type, params JSONB, enabled)	없음
만료 알림	coupon_notifications(coupon_id, type, sent_at)	없음
확장 조건	없음 (레지스트리)	없음
통계 대시보드	coupon_policy_daily_stats(policy_id, stat_date, issued, used, discount_sum)	없음
타겟팅 발급	coupon_policy_audiences(policy_id, segment_type, segment_ref)	없음
PARTNER / CAMPAIGN	coupon_policy_funders(policy_id, funder_type, funder_id, share_rate)	issuer_type CHECK 값 추가, coupon_issuer_limits·coupon_stacking_rules 행 추가
스태킹 3장+	없음	coupon_stacking_rules 행 추가 (인덱스 변경 없음)

